PROYECTO PARA EL HACKATÓN DE RECURSIVE SELF-IMPROVEMENT

PASTELES

Agentes personales y mejora recursiva verificable

Un vendedor conversacional que evoluciona su harness y las skills que construyen a su sucesor.

Resumen. Pasteles es una demostración privada de venta de pasteles por Telegram. Un agente personal conserva preferencias del cliente; un agente vendedor recibe únicamente los datos autorizados para la conversación. Un modelo pequeño interpreta solicitudes, mientras el código calcula precios, verifica condiciones y controla pagos simulados.

El ciclo RSI evalúa una versión, propone un cambio acotado a su harness y actualiza la skill del constructor. Una compuerta independiente decide si aceptar el sucesor. La versión aceptada construye la siguiente generación, con linaje, registro de rechazos y posibilidad de volver atrás. Este informe añade cuatro ejemplos de conversación y la medición de una corrección del diálogo motivada por una captura real del equipo.

495	**6/15 → 13/15**	**7/15 → 15/15**
Pruebas de software aprobadas	RSI: primer sucesor con Claude real	Experiencia: nueva evaluación con Claude real

Alcance de la evidencia. Son instrumentos distintos sobre guiones sintéticos. El catálogo y los pagos son ficticios. La reserva histórica alcanzó 38/45 objetivos y corresponde al código anterior; la corrección reciente aún no tiene una reserva independiente. No se presentan propuestas como ventas.

Corte documental: 9 de noviembre de 2026, 18:07 de Monterrey

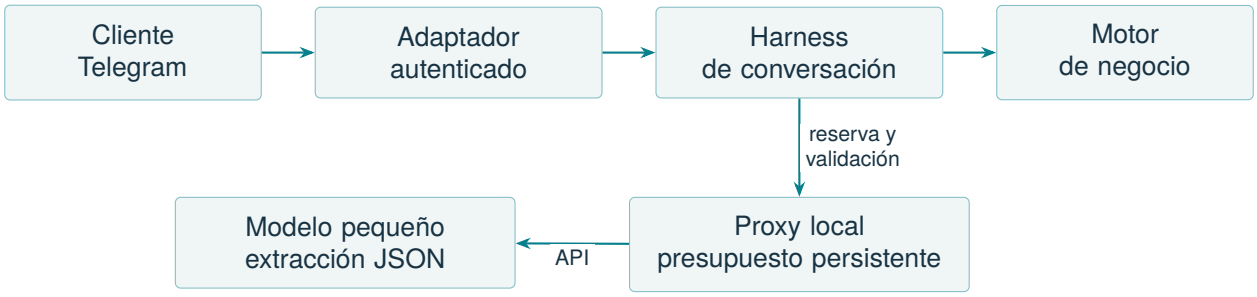
Código de referencia: `a242a35`, fases 5 a 9 registradas

Gasto acumulado al corte: USD 0.045687 de un máximo de USD 5

Guía de lectura: arquitectura y RSI en las secciones 1–4; ejemplos y nueva medición en 5–7; negocio, seguridad, panel y operación en 8–12.

1 Del mensaje a una cotización verificable

El problema inicial fue concreto: el modelo podía omitir una nueva cotización después de un cambio de fecha o presupuesto. Reforzar el prompt mejoró parte de la conducta, pero no garantizó que se invocara la herramienta. El nuevo harness lleva esa decisión a una máquina de estado explícita.



Esquema lógico: toda inferencia del modelo pasa por el proxy y la frontera LLM.

Responsabilidades separadas

Componente	Qué hace
Adaptador	Autentica usuario y chat, serializa mensajes, consume comandos y evita que un error se delegue al agente general.
Modelo	Extrae intención y campos permitidos: sabor, asistentes, fecha, entrega, restricciones y presupuesto.
Harness	Conserva pedido y preguntas pendientes, decide qué falta, invalida la cotización anterior y ejecuta la política versionada.
Negocio y pagos	Calculan importes en centavos, aplican disponibilidad y restricciones, y registran decisiones humanas de simulación.

Un turno ordinario

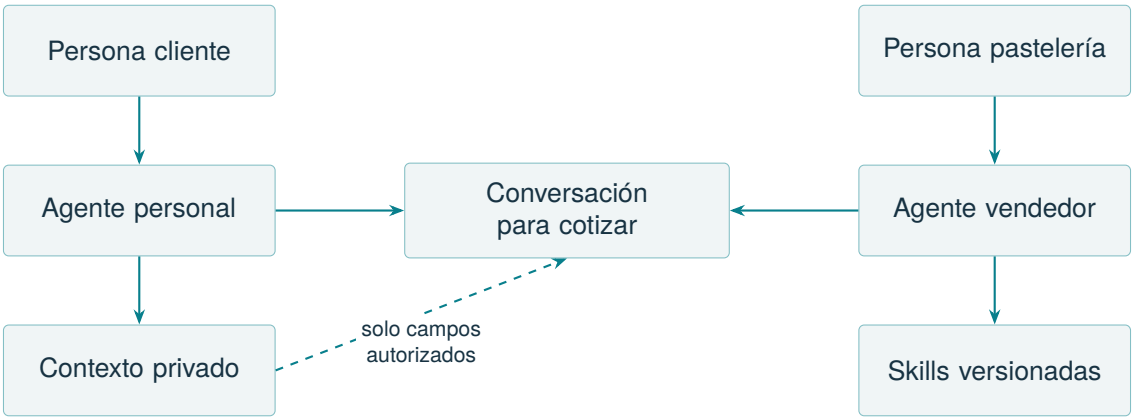
1. Se invalida el pago anterior antes de esperar una respuesta del modelo.
2. Se solicita una extracción JSON, con hasta 768 tokens de salida. El extractor recibe pedido, preguntas pendientes y última respuesta acotada; una entrada inválida borra el estado ambiguo y obliga a recoger los datos de nuevo.
3. El harness combina los datos del turno con las preferencias autorizadas que permita la versión; los datos actuales tienen prioridad.
4. Se comprueba la disponibilidad y el motor produce una cotización nueva cuando hay datos suficientes. El texto final y sus importes se construyen a partir de ese resultado; el modelo no aprueba el pago.

Ejemplo. Chocolate para 10 personas, entrega en centro: pastel de $650 MXN y entrega de $60 MXN, total de $710 MXN. Si el presupuesto cambia a $700, el motor devuelve `excede_presupuesto` y deshabilita el pago.

Fuente: `pasteles/arnes.py`, `pasteles/negocio.py` y `pasteles/telegram_rsi.py`.

2 Ontología: personas, agentes y conversaciones

La ontología es el vocabulario compartido que permite identificar quién es dueño de un dato, qué agente puede actuar y qué significa cada mensaje. Se implementa como contratos tipados y un enrutador; los permisos se establecen fuera del texto que genera el modelo.



Entidad	Contrato
Persona y agente	Identidad opaca, propietario, rol, capacidades y referencias a skills. La autenticación procede del adaptador.
Contexto	Preferencias estructuradas de un cliente. Guardarlas y compartirlas son permisos distintos.
Conversación	Participantes y finalidad explícita: `cotizar_pastel`.
Autorización	Concesión para campos concretos, origen, destino y conversación determinados.
Sobre	Mensaje con origen, destino, tipo, datos y correlación. No crea permisos ni reemplaza una skill.

Tres mensajes con significado preciso

`pedido` comunica la solicitud del agente personal al vendedor. `cotizacion` devuelve solicitud y resultado vinculados por correlación; el enrutador comprueba que coincidan con el motor de negocio. `preferencias_compartidas` transporta únicamente el subconjunto concedido.

Memoria para un cliente frecuente

Se admiten sabor, modalidad, zona e idioma mediante valores enumerados. No se guardan instrucciones libres, precios, fechas ni restricciones alimentarias en esta memoria. Los archivos tienen nombres opacos derivados con HMAC, contenido firmado y permisos privados; el contexto de un cliente no se entrega al constructor.

`/recordar sabor=chocolate modalidad=recoger` autoriza guardar.
`/compartir sabor modalidad` autoriza compartir en esa conversación.
`/new` revoca la concesión; `/olvidar` borra las preferencias propias.

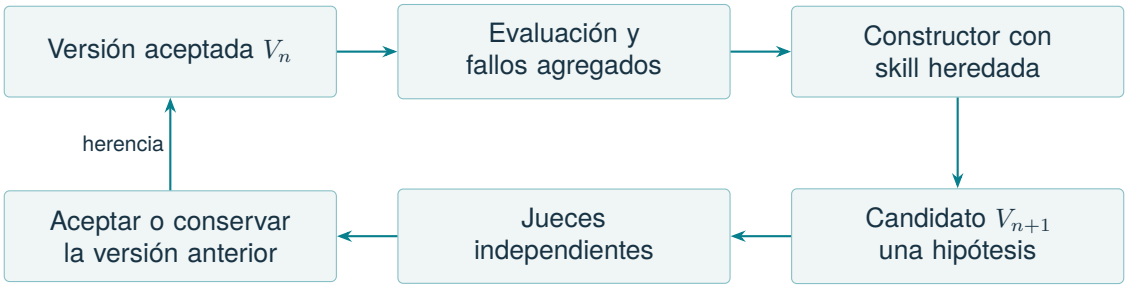
Fuente: `pasteles/ontologia.py`, `pasteles/contexto_clientes.py` y `docs/ONTOLOGIA.md`.

3 Cómo funciona la mejora recursiva

La unidad que evoluciona es una versión del sistema de conversación. El modelo base permanece fijo. Cada versión incluye una política ejecutable del harness, una skill de vendedor y una skill de constructor:

$$V_n = (H_n,\, S_n^{\text{vendedor}},\, S_n^{\text{constructor}}).$$

El constructor utiliza la versión aceptada y resultados agregados de desarrollo para producir un candidato. Su propia skill actualizada pasa al sucesor; por eso la siguiente generación no parte otra vez del constructor inicial.



Un espacio de cambios acotado

Un candidato modifica exactamente una decisión: cotización automática, uso de preferencias compartidas, una o dos preguntas por turno, cierre consultivo o estrategia comercial. Debe actualizar la skill del constructor; también puede revisar la del vendedor. La salida es un parche JSON que el orquestador ensambla y valida. No se ejecuta código generado. Si el JSON es inválido, se permite una sola corrección de formato, con el mismo presupuesto.

Compuerta de aceptación

La decisión compara comprobaciones pareadas bajo el mismo instrumento:

$$A = E \wedge O \wedge (S = 1) \wedge (R = 0) \wedge (M > 0).$$

Aquí E significa evaluación íntegra y completa; O, costo y latencia dentro de los criterios; S, proporción de controles de seguridad aprobados; R, número de regresiones; y M, mejoras funcionales o comerciales. Una mejora comercial puede acompañarse de empate funcional, pero no compensar una regresión.

> **Límite de autoridad.** El candidato no puede modificar juez, corpus, umbrales, presupuesto, modelo, permisos ni herramientas. Un empate detiene la ronda. Los resultados negativos se conservan y la versión anterior sigue activa.

El manifiesto vincula contenido y padre mediante huellas. La bitácora identifica propuestas, decisiones y publicaciones; el rollback admite versiones iniciales o previamente aceptadas.

Fuente: `pasteles/constructor_rsi.py`, `pasteles/orquestador_rsi.py` y `pasteles/versiones_rsi.py`.

4 Evaluaciones y evidencia observada

Los instrumentos separan comportamiento del harness, uso del modelo y operación del canal. Desarrollo sirve para proponer sucesores; la reserva observa el comportamiento en casos apartados y no se transmite al constructor. Los hashes fijan corpus, criterios y jueces; revisiones diferentes no se comparan como el mismo experimento.

Evidencia	Resultado	Qué permite afirmar
Suite de software actual	495 pruebas aprobadas	Contratos y regresiones cubiertos por pruebas; no son 495 conversaciones reales.
RSI determinista	13/18 → 18/18; seguridad 119/119	Mejora de política con reglas y extracciones preparadas.
Sucesor real	6/15 → 13/15; seguridad 119/119	Siete mejoras pareadas, sin regresiones, con propuesta y extracción mediante Claude.
Reserva RSI histórica	36/36 conversaciones; 38/45 objetivos; seguridad 357/357	Casos apartados del desarrollo, con siete objetivos incumplidos; evaluó el código anterior.
Comercial sintética	Cierre 0/5 → 5/5; valor 0/2 → 2/2; seguridad 61/61	Preparación del cierre y aprovechamiento de oportunidades definidas.

Fuente: `docs/VERIFICACION.md` y reportes agregados indicados en la sección 12.

El historial incluye interrupciones y rechazos

1. El ciclo determinista aceptó cuatro generaciones y rechazó la quinta por no mejorar. Su constructor fue de reglas: demuestra el mecanismo del ciclo.
2. El primer ciclo real publicó un sucesor, pero terminó interrumpido al fallar el formato JSON de una propuesta posterior.
3. El segundo intento real se interrumpió antes de publicar. Después de reducir la propuesta a un parche pequeño, un tercer ciclo evaluó y rechazó un empate.

La versión real aceptada habilita recotización automática. Las políticas de preferencias automáticas, cierre consultivo y mayor valor todavía no fueron promovidas por el ciclo real.

Costo, latencia y lectura de los denominadores

La reserva consumió **USD 0.0058074**, con latencia p95 de **3.79 segundos** por caso. Al cerrar esa etapa, el gasto fue **USD 0.0413926 de 5**, sin reservas pendientes. Tras la corrección de experiencia y el uso intermedio del bot, el acumulado llegó a **USD 0.045687**; se detalla la nueva medición en la sección 7.

> Los 18 objetivos deterministas y los 15 objetivos reales tienen distinto alcance: tres oportunidades corresponden a entradas estructuradas inválidas. No deben compararse como muestras equivalentes. Tampoco los 495 tests ni el 100 % de los controles cubiertos demuestran superioridad estadística o inmunidad universal.

5 Ejemplos: intención y respuestas breves

Una captura de Telegram mostró fallos que los benchmarks previos no explicaban bien: un «no» alimentario cancelaba la compra y una petición informal recibía una respuesta de seguridad. Se reprodujeron en un instrumento nuevo.

Cómo leer estos ejemplos. La captura motivó el cambio. Los diálogos de estas dos páginas son **extractos de reproducciones sintéticas con Claude real**, antes y después de la corrección; no son una continuación del chat privado. Se omiten el prefijo repetido de simulación y detalles sin relación con el fallo.

1. «No» responde a una pregunta; no cancela automáticamente

Situación: chocolate para cinco personas, jueves, entrega en centro. El único dato pendiente es si existe una restricción alimentaria.

ANTES · CLAUDE REAL

Bot ¿Hay alguna restricción alimentaria?
Cliente no
Bot De acuerdo. Gracias por visitarnos.
Resultado: se pierde el pedido.

DESPUÉS · CLAUDE REAL

Bot ¿Hay alguna restricción alimentaria?
Cliente no
Bot Chocolate intenso · 12 porciones para 5 personas. Total: $710.00 MXN.
Resultado: conserva el pedido y cotiza.

Qué cambió. El harness recuerda la pregunta pendiente y pregunta por restricciones por separado. Una negación mínima ante esa única pregunta significa «ninguna». «No gracias, cancela el pedido» sigue siendo una cancelación.

2. Los errores de escritura no son una instrucción maliciosa

ANTES · CLAUDE REAL

Cliente tengo ambre
Bot Puedo ayudarte con el catálogo y una cotización. No puedo cambiar reglas ni acceder a datos de otras personas.

DESPUÉS · CLAUDE REAL

Cliente tengo ambre
Bot ¿Qué sabor prefieres? ¿Para cuántas personas?

Qué cambió. El contrato de extracción distingue una expresión informal de interés de un intento de cambiar permisos, precios o reglas. La variante «vendene un pastel» también recibió preguntas comerciales en la medición corregida.

Fuente: Experiencia v1, casos `continuidad_no` y `coloquial_hambre`; reportes reales de la sección 12.

6 Ejemplos: disponibilidad y orientación de pago

La conversación debe resolver primero el impedimento relevante. Pedir todos los datos de un producto agotado o rechazar una pregunta de pago añade fricción sin ayudar al cliente a decidir.

3. Comprobar el catálogo antes de seguir preguntando

ANTES · CLAUDE REAL

Cliente quiero un pastel de fresa

Bot ¿Para cuántas personas? ¿Para qué fecha lo necesitas?

Resultado: solicita más datos sin advertir todavía que fresa está agotada.

DESPUÉS · CLAUDE REAL

Cliente quiero un pastel de fresa

Bot Ese pastel no está disponible en el catálogo de hoy. Tenemos: Vainilla clásica (12 porciones), Chocolate intenso (12 porciones), Chocolate para compartir (20 porciones). ¿Cuál te gustaría?

Qué cambió. La disponibilidad se comprueba cuando ya se conoce el sabor o producto. El sistema ofrece alternativas disponibles y espera una nueva elección; no sustituye silenciosamente el pedido ni habilita un pago por un producto agotado.

4. Explicar cómo pagar sin inventar una cotización

Situación: el cliente pregunta por el pago sin haber completado un pedido.

ANTES · CLAUDE REAL

Cliente como pago?

Bot Puedo ayudarte con el catálogo y una cotización. No puedo cambiar reglas ni acceder a datos de otras personas.

DESPUÉS · CLAUDE REAL

Cliente como pago?

Bot Antes de usar /pagar necesitamos una cotización completa y vigente. Los botones solo simulan el pago; no se cobra dinero.

Bot ¿Qué sabor prefieres? ¿Para cuántas personas?

Qué cambió. La intención `consulta_pago` forma parte del contrato. Si faltan datos, orienta y continúa preguntando. Si el pedido está completo, recalcula una cotización vigente antes de indicar `/pagar`; preguntar no aprueba un pago ni reactiva una tarjeta antigua.

Con pedido completo, también se verificó: «Usa /pagar para abrir los botones y elegir aprobar, rechazar o cancelar el pago simulado. Esta cotización se acaba de verificar». El total volvió a calcularse en ese turno.

Fuente: Experiencia v1, casos `fresa_temprana`, `pago_sin_datos` y `continuidad_no`.

7 Del fallo observado a una medición nueva

La causa principal fue una pérdida de contexto: el extractor recibía el pedido acumulado, pero no la pregunta que el bot acababa de formular. La corrección añadió preguntas pendientes y la última respuesta, limitada a 2 000 caracteres, además de disponibilidad temprana y orientación de pago.

Experiencia v1: mismo instrumento, dos revisiones del código

Se definieron **ocho conversaciones públicas sintéticas**, con **15 objetivos y 120 controles**. Se congelaron casos, criterios y juez antes de medir. Modelo y versión declarativa fueron iguales; cambió el código del harness. Cada modalidad se ejecutó una vez por revisión.

Medida	Anterior	Corregido
Objetivos con JSON preparado	11/15	15/15
Objetivos con Claude real	7/15	15/15
Controles, en ambas modalidades	120/120	120/120
Conversaciones reales completas	8/8	8/8
Costo de la corrida real, USD	0.0014839	0.0020912
Latencia p95 por conversación	2.53 s	2.61 s



Objetivos cumplidos con Claude real. Escala común de cero a quince; sin banda de significancia estadística.

El evaluador también debe detectar errores

El juez contrasta estado, preguntas, solicitud, cotización y pago. Se prueba que rechace cotizaciones antiguas y que un promedio favorable no oculte una regresión. Una interrupción conserva objetivos pendientes como fallos. La prueba offline usa JSON preparado; la real exige que Claude interprete los mensajes.

Las dos corridas reales costaron **USD 0.0035751** en conjunto. El acumulado fue **USD 0.045687 de 5**, sin reservas pendientes. La suite completa pasó **495 tests**. El bot estuvo detenido durante la medición y se reinició con la corrección a las 18:07 de Monterrey.

Qué demuestra y qué falta. Es una corrección de ingeniería, no una promoción autónoma RSI. El 15/15 corresponde a desarrollo público con una repetición; no hay reserva independiente de este código. El juez comparte el motor de negocio con la aplicación. La prueba no ejecuta Telegram ni decisiones humanas de pago, y no acredita naturalidad, satisfacción, conversión o ventas.

Fuente: `evaluacion_experiencia/README.md`, `pasteles/evaluacion_experiencia.py` y `docs/VERIFICACION.md`.

8 Estrategia comercial para un modelo pequeño

El vendedor busca facilitar una decisión informada y aumentar el valor de opciones pertinentes. La estrategia combina descubrimiento progresivo, comparación de beneficios, reducción de incertidumbre y una invitación clara al siguiente paso. El cliente conserva la decisión de aprobar, rechazar o cancelar la simulación.

1. **Descubrir la necesidad.** Obtener fecha, asistentes, sabor, restricciones, modalidad y presupuesto, sin repetir información ya autorizada.
2. **Explicar la opción base.** Mostrar porciones, precio completo y entrega. La cotización debe corresponder al motor de negocio.
3. **Proponer más valor.** Si el cliente quiere porciones extra y tiene un presupuesto explícito suficiente, comparar una opción mayor con beneficio e incremento de precio concretos.
4. **Preparar el cierre.** Ante una cotización viable, preguntar si desea probar esa opción e indicar `/pagar`. Un rechazo cierra el intento.
5. **Medir y aprender.** Registrar oportunidades, decisiones y resultados; proponer una sola hipótesis por generación y conservar las que fallen.

Dos políticas comerciales

Ajuste elige la opción de menor precio que satisface la solicitud. **Valor** puede ofrecer una opción de más porciones si el cliente las desea y el total, incluida la entrega, cabe en el presupuesto. La selección busca el mayor importe entre esas alternativas autorizadas:

$$p^* = \arg\max_{p \in \mathcal{F}} \text{total}(p), \qquad \text{total}(p) \leq B.$$

$\mathcal{F}$ contiene únicamente alternativas disponibles, suficientes, compatibles con sabor, fecha, restricciones y preferencia de porciones extra; B es el presupuesto declarado. Si la opción base ya supera B, el sistema conserva el impedimento visible y no habilita el pago.

> **Ejemplo de propuesta.** Ocho porciones adicionales por $300 MXN más, con total de entrega incluido. La explicación muestra qué obtiene el cliente y cuánto cuesta; el catálogo demo no aporta evidencia para argumentos de escasez, urgencia o reseñas.

Qué significa maximizar ventas en esta demo

En **dos escenarios sintéticos**, el ticket potencial medio pasó de **$695 a $995 MXN**. Esta métrica describe propuestas, no compras ni una probabilidad estimada de conversión. En la versión real activa esas políticas comerciales avanzadas aún no se han promovido.

El embudo observado usa exportaciones de pagos simulados. Calcula aprobaciones sobre todos los intentos y sobre los resueltos, ingreso simulado y ticket de los aprobados. Rechazos, cancelaciones, caducados e invalidados permanecen visibles; solo los pendientes se excluyen del denominador de resueltos. Cero oportunidades produce una proporción no aplicable, nunca un éxito ficticio.

Fuente: `pasteles/ventas.py`, `pasteles/evaluacion_ventas.py` y `docs/ESTRATEGIA_COMERCIAL.md`.

9 Seguridad y protección del experimento

Las defensas separan datos de autoridad. La entrada de un cliente puede expresar una solicitud, pero no conceder un rol, editar una skill, cambiar el catálogo ni aprobar un pago. El modelo no recibe una herramienta de escritura sobre el sistema ni los archivos privados de otros clientes.

Riesgo	Control implementado
Suplantación de identidad	Origen autenticado por Telegram; agente y permisos registrados por el orquestador, no aceptados desde el modelo.
Instrucciones en datos	Extracción con esquema cerrado, contexto enumerado y sobres que rechazan campos ajenos al contrato.
Precio o pago inventado	Importe calculado por el motor; botón vinculado a usuario, chat, mensaje y generación vigentes.
Cotización obsoleta	Invalidación antes de inferir; nueva cotización al cambiar datos; ediciones y resultados tardíos no reviven la tarjeta.
Confusión visual	Texto decorativo citado con controles de formato escapados, separado de precios y estados del sistema.
Fuga de memoria	Archivos privados por cliente, validación de enlaces y contenido; concesión explícita para cada subconjunto compartido.
Manipulación de evals	Juez y criterios fuera de las versiones editables; hashes, reportes históricos y reservado excluido del constructor.

Presupuesto como estado persistente

Toda inferencia pasa por `pasteles/llm.py`. La cuenta reserva capacidad antes de llamar al proveedor y liquida el consumo cuando lo conoce. Una llamada incierta conserva su reserva. Reiniciar el programa no reinicia el saldo, y el tope de **USD 5 acumulados** incluye conversaciones, evaluación y construcción.

La clave de Anthropic se recibe por entorno y permanece en el proceso del proxy; Hermes usa una credencial local distinta. El proxy escucha en loopback. Los tests usan clientes falsos y no requieren llamadas a la API.

Fallos tratados como parte del diseño

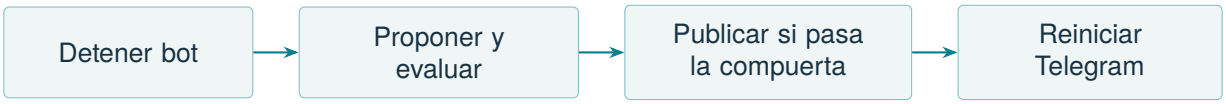
Un JSON inválido, una cuota agotada, una respuesta tardía o una evaluación incompleta detienen la operación correspondiente y conservan la evidencia. El adaptador consume el mensaje incluso al fallar: no lo transfiere a un agente general con capacidades más amplias. La publicación registra la aceptación antes de cambiar el puntero de la versión activa.

Límite de la garantía. Las pruebas cubren ataques y fronteras concretas. La comprensión lingüística puede fallar y quedan objetivos pendientes en la reserva. La seguridad del prototipo no equivale a inmunidad frente a toda prompt injection ni a aprobación para atender clientes reales.

Fuente: `docs/RSI.md`, módulos de contexto, ontología, presupuesto y transporte, y sus tests.

10 Cuándo cambia el agente en operación

Actualmente el ciclo RSI real se inicia de forma explícita. El bot y la evaluación comparten un bloqueo: primero se detiene Telegram, se ejecuta el ciclo y después se reinicia el servicio con la última versión aceptada. No existe un disparador automático por número de mensajes o ventas.



Cada conversación carga una versión al crearse y la conserva mientras existe. Un candidato aceptado pasa a ser el padre de la generación siguiente, pero el cliente verá los cambios cuando el bot vuelva a crear la conversación. El reinicio invalida las cotizaciones anteriores. Un rechazo posterior no elimina al último sucesor ya aceptado.

Qué cambia	Disparador
Contexto personal	Comandos explícitos del cliente para guardar, compartir o borrar.
Política y skills	Candidato aprobado y publicación en la línea real.
Código base del harness	Corrección de ingeniería verificada por separado, con identificación de la revisión evaluada.
Juez, permisos y presupuesto	Permanecen fuera de la autoridad del constructor RSI.
Modelo base	Permanece fijo en este experimento; no hay ajuste de pesos.

Publicación y reserva son pasos distintos. La promoción actual usa desarrollo y comprobaciones comerciales. La reserva se ejecuta aparte; todavía no es una condición obligatoria previa a publicar. El entorno activo es una demo privada y la aceptación empírica no constituye validación comercial final.

Estado de la versión documentada

El sucesor real `b37113db...` habilita cotización automática. La memoria y los permisos para compartir están disponibles, pero aplicar automáticamente esas preferencias depende de la política aceptada. El cierre consultivo y la estrategia de mayor valor están implementados y probados en la línea de simulación; no se fuerza su promoción a la línea real para presentar mejores cifras. La revisión `a242a35` conserva esa política y añade la corrección contextual descrita en los ejemplos. El panel identifica tanto la versión declarativa como el SHA del código: la reserva anterior no valida automáticamente esta revisión.

Próximos pasos

Para evolucionar hacia un servicio público harían falta validación de experiencia con personas, una condición explícita de aceptación final previa al despliegue y observación del embudo real. Si los fallos reservados actuales se utilizan para diseñar otra mejora, se necesitará una nueva reserva independiente. Estas tareas se mantienen separadas de los entregables ya implementados.

Fuente: `pasteles/__main__.py`, `pasteles/orquestador_rsi.py` y `pasteles/telegram_rsi.py`.

11 Dónde se puede ver la mejora

El dashboard local reúne evidencia agregada, versiones y decisiones. Se abre con `./PANEL.sh` en http://127.0.0.1:8765. Es de solo lectura: no llama al modelo, no acepta candidatos ni modifica el presupuesto.

Vista	Qué muestra	Cómo interpretarla
Modelo real	Corridas con llamadas a Claude y sus costos.	Los mensajes siguen siendo guiones sintéticos; no son indicadores de clientes en producción.
Simulación	Extracciones preparadas y ciclo con constructor de reglas.	Demuestra lógica y controles, no comprensión lingüística.
Experiencia	Código anterior: 7/15; corregido: 15/15, en la línea real.	Nuevo instrumento de desarrollo, derivado de fallos observados.
Reserva	Resultado histórico de 38/45, junto a la revisión evaluada.	No valida un código que cambió después; falta reserva de la revisión actual.
Historial	Publicaciones, rechazos e intentos interrumpidos.	Una corrida interrumpida puede conservar un sucesor publicado previamente.

Dos identidades para no atribuir una mejora al código equivocado

La versión declarativa identifica política y skills. El SHA del harness identifica el código que las ejecuta. Ambos deben coincidir para señalar que una reserva corresponde a la implementación actual. La corrección de los ejemplos cambió el segundo, manteniendo la política RSI aceptada.

Leer una medición. En la tabla de experiencia, comprobar tipo de ejecución, revisión del código, objetivos, controles y conversaciones completas. Después, consultar el instrumento y los criterios. «15/15» sin esos datos no permite saber qué se ejecutó ni si el resultado corresponde al bot actual.

Actualizar la vista no vuelve a evaluar al agente

El panel vuelve a leer archivos cada 30 segundos. Nuevas puntuaciones aparecen cuando se ejecuta otra evaluación; los mensajes de Telegram no se puntúan automáticamente. El gasto sí puede cambiar si el bot sigue atendiendo mensajes, porque todas las llamadas usan la misma cuenta acumulada.

Para repetir únicamente el instrumento de experiencia:

```
.venv/bin/python -m pasteles experiencia --modo offline
.venv/bin/python -m pasteles experiencia --modo real
```

La segunda orden necesita el bot detenido y la clave en el entorno; consume presupuesto. Repetir desarrollo no lo convierte en un conjunto reservado.

Fuente: `docs/PANEL.md`, `pasteles/panel_datos.py` y `pasteles/panel_experiencia.py`.

12 Reproducción y trazabilidad

Los comandos se ejecutan desde la raíz del proyecto. La simulación conserva una línea independiente; no publica sus versiones en el bot real. Las llamadas reales comparten el presupuesto ya consumido.

```
make verificar
./RSI.sh simulacion 5
./RSI.sh real 4
.venv/bin/python -m pasteles rsi estado --modo real
.venv/bin/python -m pasteles configurar --rsi
./INICIAR.sh
```

El bot debe detenerse antes del ciclo. El script real pide la clave de manera oculta si no existe en el entorno; no debe incluirse en argumentos, documentos ni repositorios. Repetir una corrida crea otro reporte; no hay que borrar la contabilidad para repetir la demo.

Mapa de artefactos

Ruta relativa	Contenido
`.runtime/rsi/versiones/`	Harness, skills y manifiestos de la línea real.
`.runtime/rsi-simulacion/`	Versiones y decisiones de simulación.
`resultados/rsi/`	Evaluaciones y ciclos reales, incluidos fallos.
`resultados/simulacion/rsi/`	Evidencia determinista claramente etiquetada.
`evaluacion_rsi/`	Instrumento RSI congelado, desarrollo y reserva.
`evaluacion_ventas/`	Instrumento comercial y snapshot de su revisión anterior.

Fuentes locales del informe

[1] `docs/VERIFICACION.md`: historial de pruebas, mediciones, límites y gasto.

[2] `docs/RSI.md`, `docs/ONTOLOGIA.md` y `docs/ESTRATEGIA_COMERCIAL.md`: contratos y decisiones de arquitectura.

[3] Simulación: `20261109T231432-83a1b09c87/reporte.json`, bajo `resultados/simulacion/rsi/`.

[4] Ciclos reales bajo `resultados/rsi/`, cada uno con su `reporte.json`:
`20261109T231515-de256961a1`,
`20261109T231814-f5b7882632`,
`20261109T232058-d5c7664649`.

[5] Reserva real: `rsi-20261109T232300-9e9be651d2/reporte.json`, bajo `resultados/rsi/`. Se utilizaron sus agregados, no los casos para diseñar mejoras.

[6] Experiencia real, bajo `resultados/experiencia/`:
`experiencia-20261110T000614-f0a5cb2464/reporte.json` (anterior),
`experiencia-20261110T000627-b3ecb5f0d7/reporte.json` (corregido).
Sus trazas públicas contienen los extractos de las secciones 5 y 6; las carpetas usan hora UTC, equivalente a las 18:06 del 9 de noviembre en Monterrey.

[7] `evaluacion_experiencia/README.md`, casos, criterios y manifiesto de congelación: protocolo de la corrección; `docs/PANEL.md`: lectura del dashboard.

Tesis del proyecto. La contribución demostrada es un sistema que puede construir, medir, heredar y rechazar cambios sobre sí mismo dentro de un contrato explícito. La evidencia comercial y de calidad mantiene su alcance visible, junto con los resultados negativos.